

OCP Hardware Fault Management Workstreams

Moderator - Drew Walton *Microsoft*

Theodros Yigzaw *Nvidia*

Vilas Sridharan *AMD*

Samer El-Haj-Mahmoud *Arm*

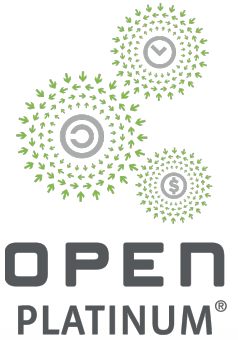
Janusz Jurski *Intel*

Carlos Vallin *Microsoft*

Shubhada Pugaonkar *Nvidia*

Marko Bartscherer *Intel*





Panel of HWFM Ecosystem Workstreams



Drew Walton

Co-Lead HWFM, FMFM and RAS API, Microsoft



Theodros Yigzaw

Co-Lead FMFM, Nvidia



Vilas Sridharan

AMD



Samer El-Haj-Mahmoud

Distinguished Engineer and System Architect at Arm



Janusz Jurski

Principal Engineer, Intel Corporation



Shubhada Pugaonkar

Platform RAS Architect, Nvidia



Carlos Vallin

Microsoft



Marko Bartscherer

Co-Lead System Debug WS, Principal Engineer, Intel Corporation

Agenda

- Intro
- Hardware Fault Management Workstreams Overview
- Progress in Hardware Fault Management
- Fleet Memory Fault Management
- RAS API Introduction
- RAS API Collaboration with DMTF Standards
  - PLDM Mapping
  - Redfish Mapping
- RAS API Proofs of Concepts
- System Debug Workstream Update
- Call to Action
- Questions

Drew Walton

OCP Hardware Fault Management Ecosystem

Ecosystem Vision

With broad industry support, we are building an ecosystem of work-streams that address some of the most challenging problems faced by large-scale data centers.

HWFM meets weekly on Fridays 2-3pm PT

Hardware Fault Management
Overarching coordinator across all other active workstreams, handles broadest topics

Fleet Memory Fault Management
DRAM-focused fault management solutions for hyperscale infrastructure

RAS API
Designing universal interfaces for logging errors and triggering automated actions

System Debug
Fostering collaboration on standard hardware and software debug architectures

Closed-CLA Working Groups Collaboration
Stay in sync with GPU, Debug, and Data Resiliency efforts, overlapping attendance

Drew Walton

HW Fault Management Progress

Major Focus Areas over the last year

HWFM Infrastructure Requirements 2.0 (Q1’27)

Error Injection Requirements

- Building common requirements to better enable critical testing and debugging

Error Severities

- Baselining error severities to CPER severities
- Provide consistency across Si providers to enable efficient and correct infrastructure implementation

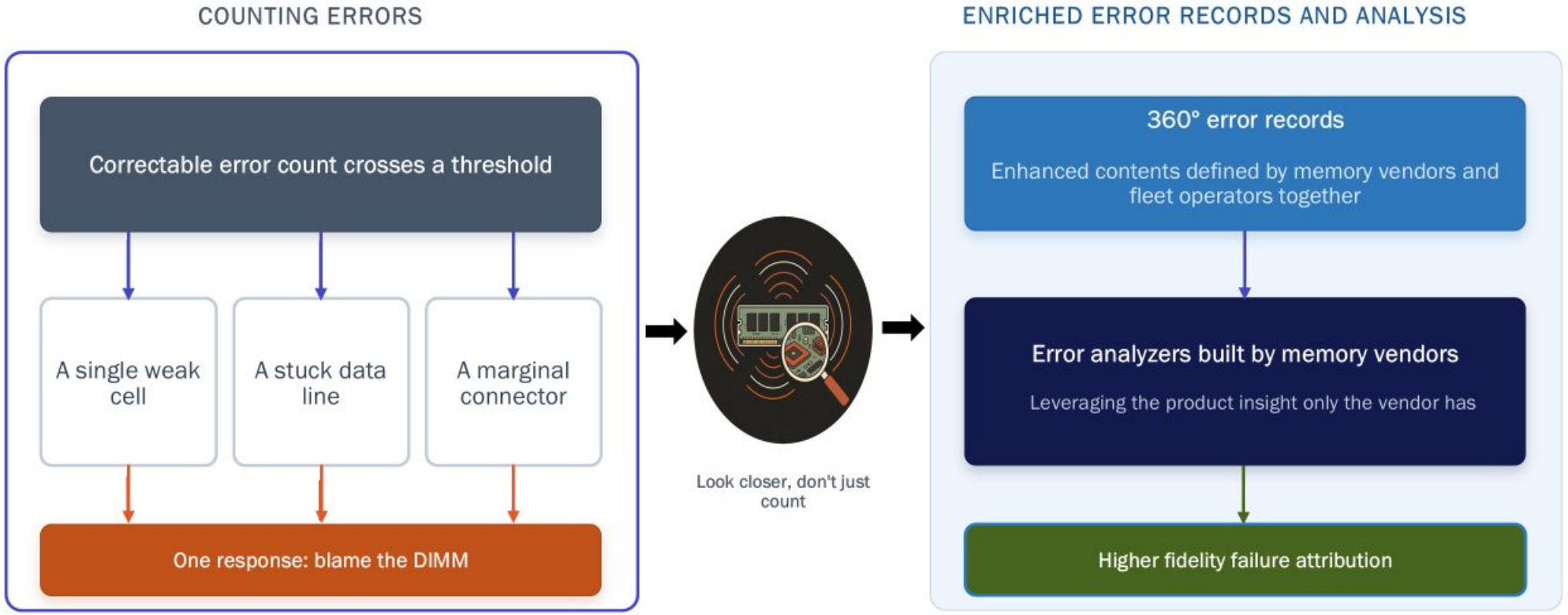
RAS API Version 0.96 (released)

- Sufficiently defined to enable DMTF (PLDM & Redfish)

FMFM workstream Progress

FMFM charter: Standardize memory fault management and RAS interfaces across vendors.

Key Insights From FMFM Work



FMFM Workstream Progress

Theo Yigzaw

FMFM WS meets bi-weekly on alternate Tuesdays 7-8am PT

Future/Ongoing Focus



01

Extend to HBM and LPDDR logging

Address divergences from DDR



02

ECS improvements

Better in-DRAM error data exposure. Starting with DDR6, applicable to LPDDR & HBM



03

Alternate internal ECC logging

Balancing IP protection vs. fleet reliability



04

RAS API alignment

Integrating analyzers with OCP RAS API. ARM booth demos with Samsung and Micron



05

Error injection requirements & security

Standard lock/unlock methods from other workstreams, with security experts



06

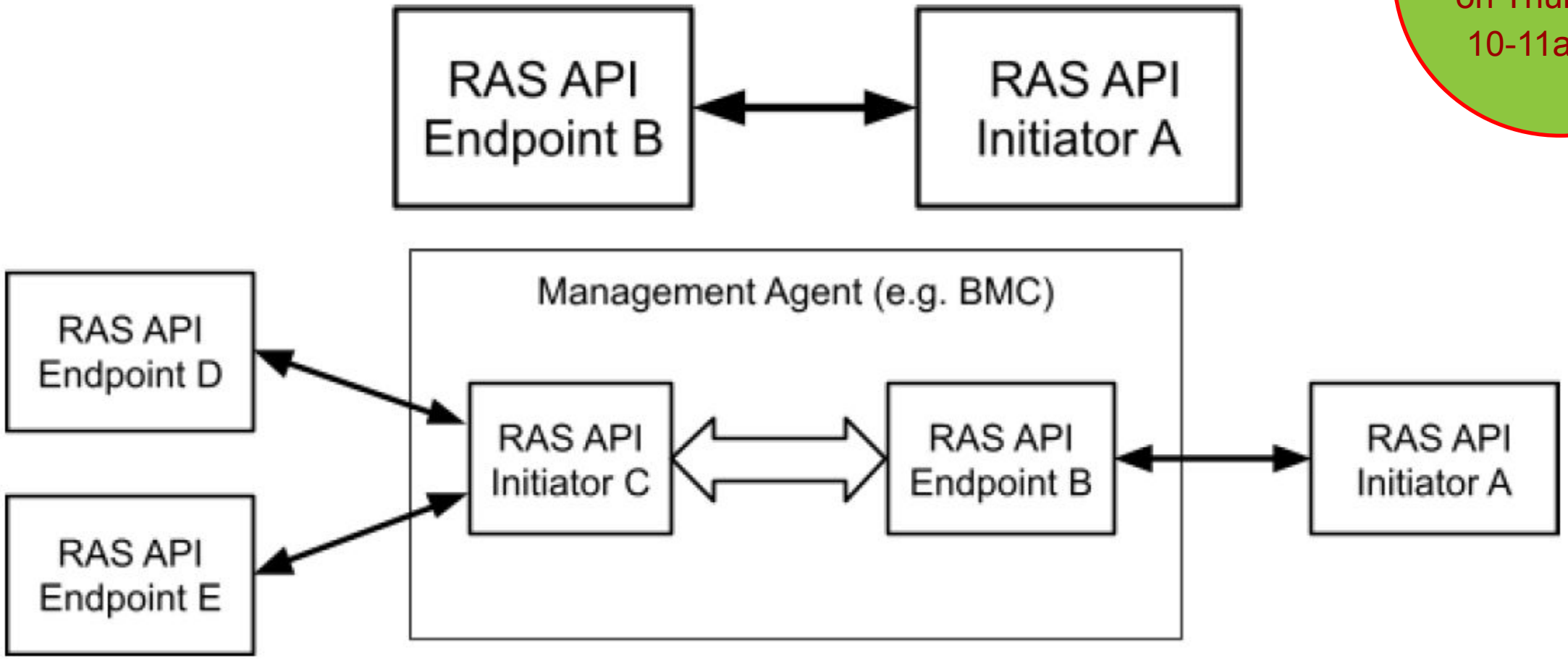
Memory scenarios white papers

Documenting best practices

RAS API Update

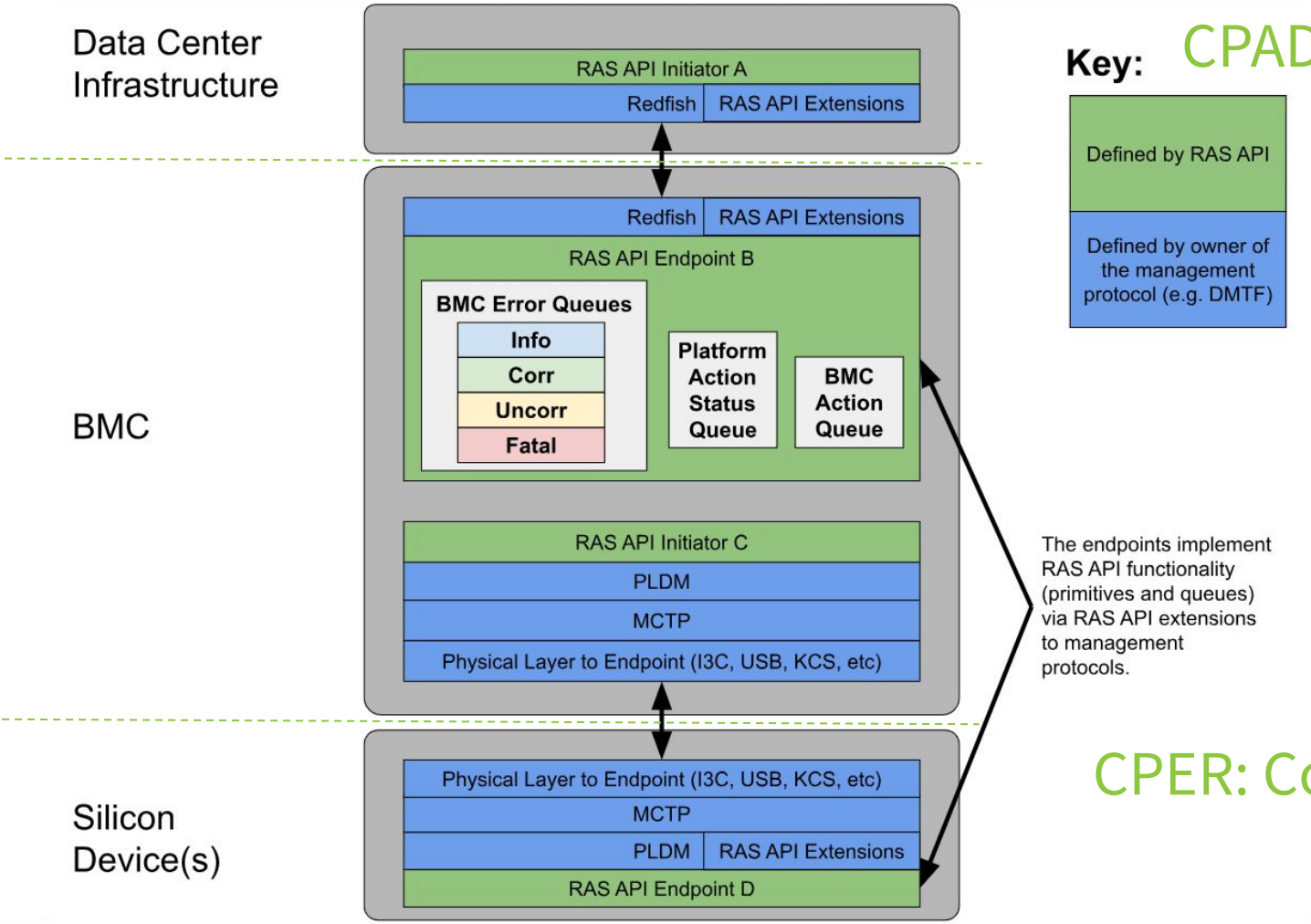
Vilas Sridharan

RAS API WS meets weekly on Thursdays 10-11am PT



RAS API Update

Vilas Sridharan



Key:

Defined by RAS API

Defined by owner of the management protocol (e.g. DMTF)

CPAD: Common Platform Action Descriptor

CPER: Common Platform Error Record

RAS API collaboration with DMTF

Samer El-Haj-Mahmoud

- RAS API Workstream defines framework and interop requirements, publishes public specs.
- Ultimately, RAS API concepts need mapping onto real world protocols, like DMTF MCTP/PLDM and Redfish, to foster adoption.
- Members who belong to both OCP and DMTF work within DMTF the evolve the PLDM and Redfish modeling to address the RAS API requirements. This work in DMTF is based on the OCP workstream's public specs.
- DMTF releases public WIP presentations and spec drafts reflecting the PLDM and Redfish modeling evolving discussions.

RAS API collaboration with DMTF

Samer El-Haj-Mahmoud

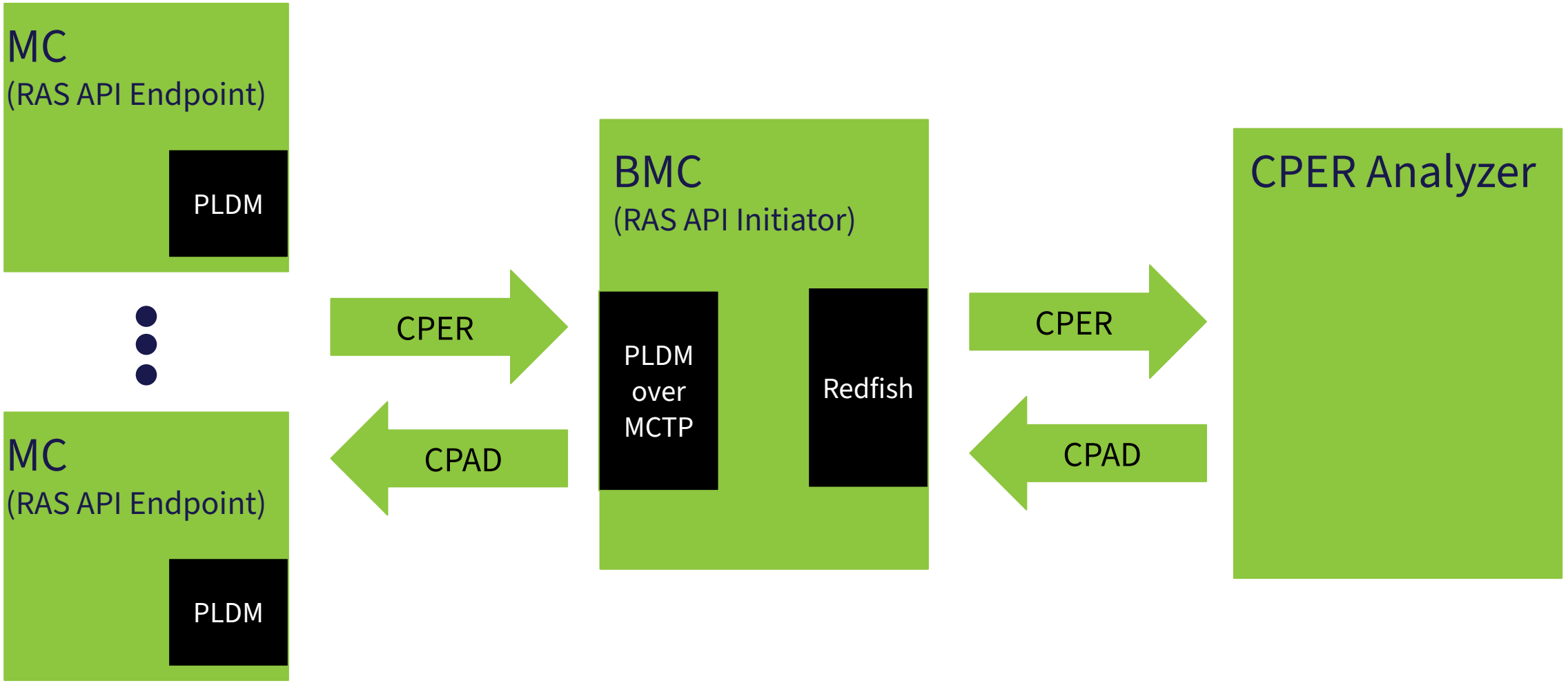
- DMTF public WIP RAS API PLDM modeling requirements: https://www.dmtf.org/sites/default/files/RAS_API_mapping_to_PLDM_requirements_WIP.pdf . Expected published white paper describing RAS API mapping to **PLDM**.

- Redfish schema proposal to be submitted to the DMTF Redfish Forum.

- Proposed PLDM and Redfish modeling aspects covered next by Janusz and Carlos.

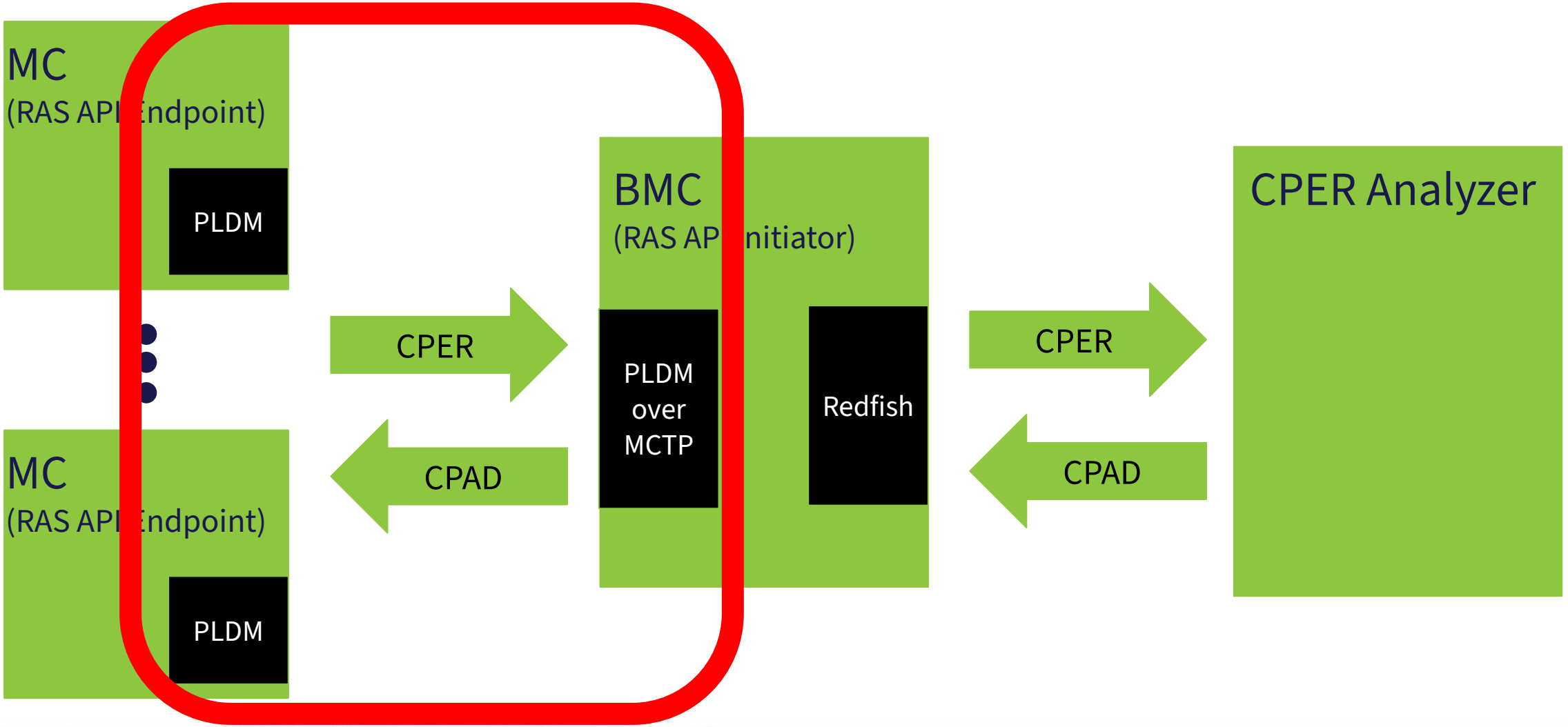
RAS API collaboration with DMTF

Samer El-Haj-Mahmoud



RAS API collaboration with DMTF

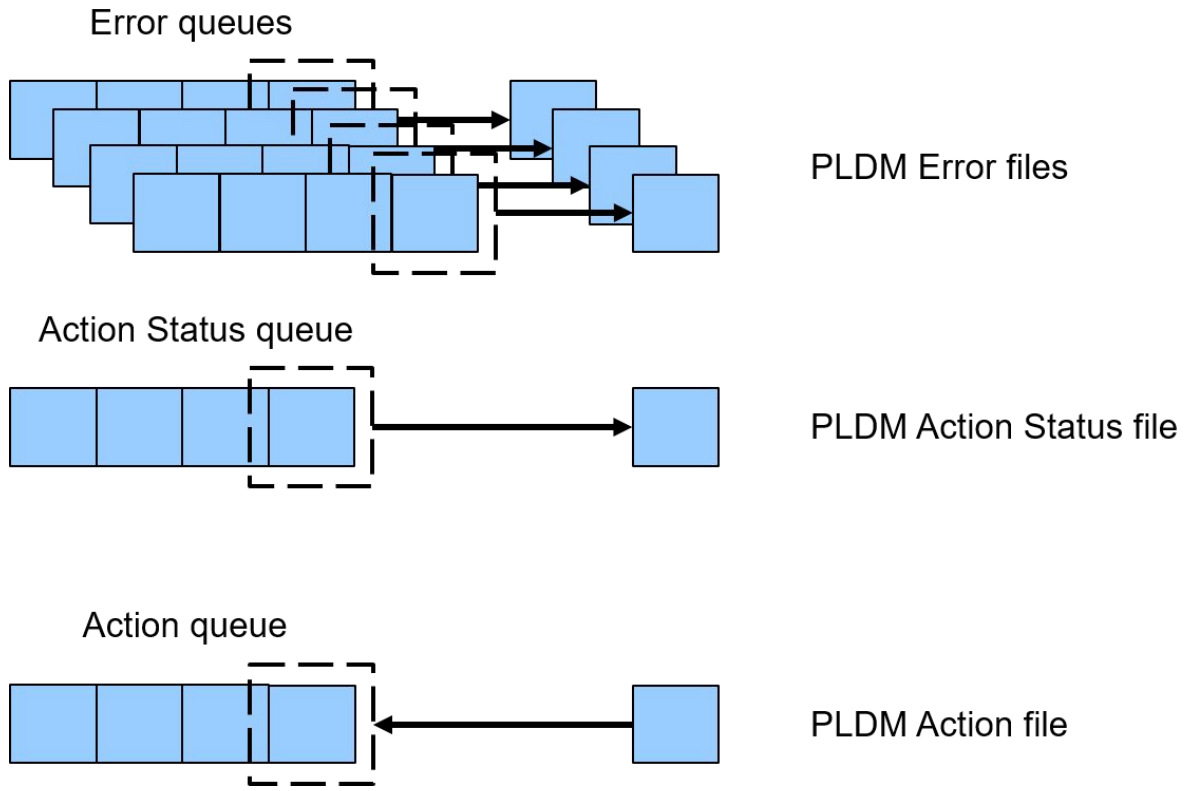
Janusz Jurski



RAS API PLDM Mapping

Janusz Jurski

- DSP2094: DMTF PMCI WG public whitepaper expected (note: requirements published by DMTF.org already)
- Based on very powerful PLDM file concept (PLDM Type 7)
  - Standard discovery mechanisms for multiple files
  - Standard I/O operations
- Common semantics:
  - Initiator (BMC) reading PLDM Error file obtains queue head entry
  - Initiator (BMC) writing PLDM Action file writes new action request



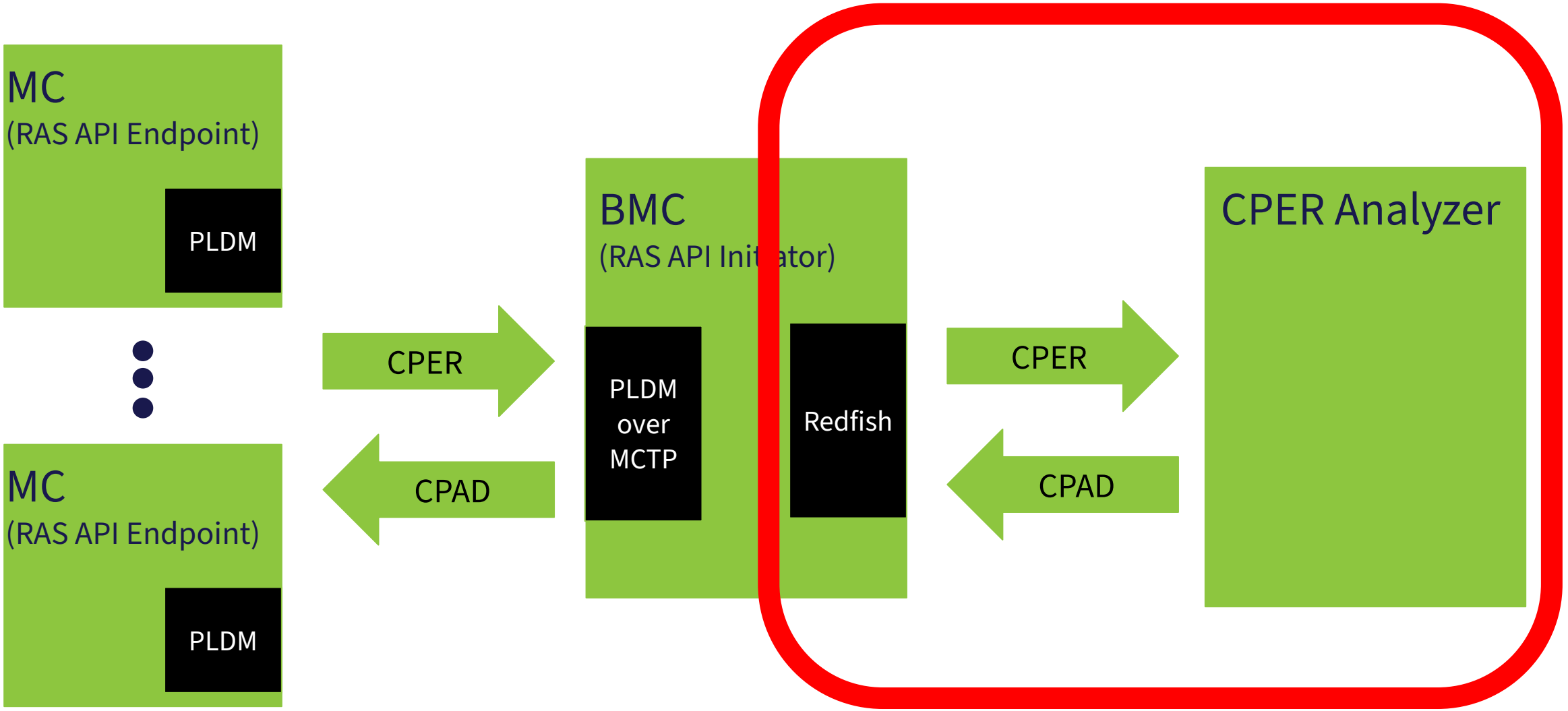
RAS API PLDM Mapping Aligned with Other Specs

- Concept common with other manageability and telemetry proposals from DMTF PMCI WG (https://www.dmtf.org/standards/pmci)
  - PLDM Bulk Telemetry Modeling (DSP2091)
  - PLDM Crash Log Modeling (DSP2092)

- All based on PLDM for File Transfer Specification (DSP0242)
  - Accompanied by PDR metadata

RAS API collaboration with DMTF

Carlos Vallin



RAS API Redfish Mapping

Carlos Vallin

- OCP RAS API Redfish Spec - WS reviewed – next step: proposal submission to Redfish Forum.
- Combines standard Redfish and a small OEM layer
  - LogService/LogEntry to collect CPERs, EventService to subscribe to CPER events
  - Oem.OCPRASAPIWS: RASService and RAS API Endpoint discovery, SubmitCPAD action

Step	Redfish operation	Return information
Discover RAS API	`GET /redfish/v1/Oem/OpenCompute_FaultMgmt/RASService`	`RASAPIVersion`; links to endpoints
Subscribe to events	`POST /redfish/v1/EventService/Subscriptions` with `RegistryPrefixes:["OCPRAS"]` or per-queue MessageIds	`201` Created; events then pushed to Destination
Collect CPERs	Client receives Redfish event message	Event message with CPER data (`DiagnosticData`) or URI pointer (`AdditionalDataURI`)
Send CPADs	`POST .../Actions/RASService.SubmitCPAD` with Base64 encoded CPAD binary	`202` Accepted; action result sent later as `PlatformActionEvent` CPER

RAS API Redfish Mapping

Carlos Vallin

Discover - does it speak RAS API?

MOCKUP

- GET */redfish/v1/* - absence of Oem.OCPRASAPIWS means not supported

```
"Oem": {
    "OCPRASAPIWS": {
        "RASService": {
            "@odata.id": "/redfish/v1/Oem/OpenCompute_FaultMgmt/RASService" } } }
```

- GET */redfish/v1/Oem/OpenCompute_FaultMgmt/RASService*

```
"RASAPIVersion": "1.0.0"
Links: RASEndpoints | CPERLogService | EventService
Actions: #RASService.SubmitCPAD
```

- GET .../*RASEndpoints/{id}* - RAS API endpoint identity
  - PartitionID (stable across BMC reboots), CreatorID, FRUID / FRUText
  - SupportedQueues: Fatal, Recoverable, Corrected, Informational, PlatformActionStatus

RAS API Redfish Mapping

Carlos Vallin

Subscribe - per-queue event filtering

- Queue -> Redfish severity -> MessageId (normative, spec section 5.2)
  - `Fatal -> Critical -> OCPRAS.1.0.0.FatalError`
  - `Recoverable -> Critical -> OCPRAS.1.0.0.UncorrectedError`
  - `Corrected -> Warning -> OCPRAS.1.0.0.CorrectedError`
  - `Informational -> OK -> OCPRAS.1.0.0.InformationalEvent`
  - `PlatformActionStatus -> Warning -> OCPRAS.1.0.0.PlatformActionEvent`
- POST /redfish/v1/EventService/Subscriptions
  - `"RegistryPrefixes": ["OCPRAS"], "ResourceTypes": ["LogEntry"]`
    `Fatal only -> "MessageIds": ["OCPRAS.1.0.0.FatalError"]`
- Fatal and Critical CPERs shall be delivered at time of occurrence - not deferred, not batched

RAS API Redfish Mapping

Carlos Vallin

Collect CPERs

- Events and LogEntry includes `"DiagnosticDataType": "CPER"`
- Pattern A - Event message contains CPER data “inline” (smaller records)
  - `"DiagnosticData": "<base64 CPER data>"`
- Pattern B **-** Event message does NOT contain the CPER data; it points to the stored LogEntry instead
  - `"OriginOfCondition": { "@odata.id": ".../CPER/Entries/123456790" }`
  - `"AdditionalDataURI": ".../Entries/123456790/Attachment"`
  - GET on that URI returns the complete binary CPER - no truncation, no re-encoding
- If polling any queues: LogEntry.Id is monotonically increasing
  - `$filter=Id gt '<last-seen-Id>'` - resume without rescan; no deletion needed

RAS API Redfish Mapping

Carlos Vallin

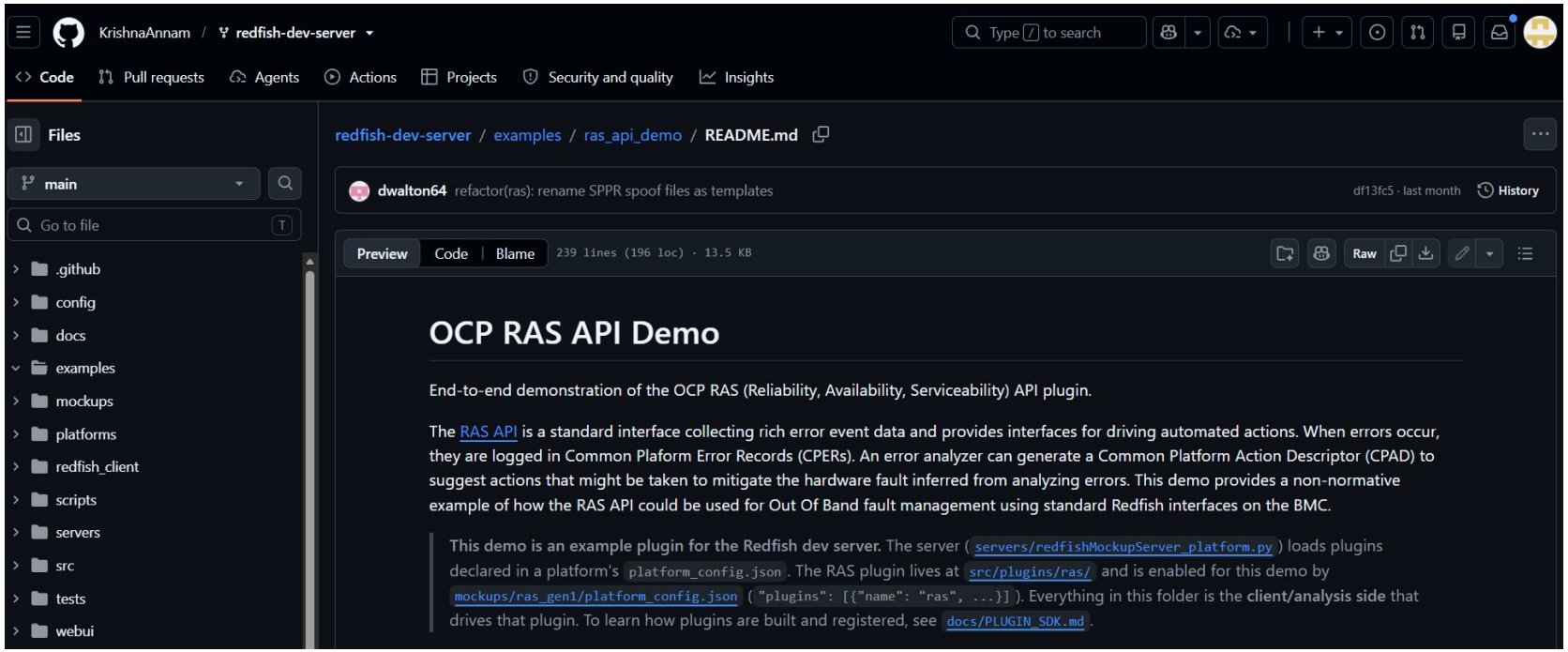
Send CPADs

- POST */redfish/v1/Oem/OpenCompute_FaultMgmt/RASService/Actions/RASService.SubmitCPAD*
  - `{ "CPADData": "<Base64-encoded binary CPAD>" }`
- BMC validates CPAD header and section descriptors; section bodies are opaque; routes by PartitionID
- Synchronous response is acceptance only:
  - **202** accepted for routing
  - **400** malformed
  - **403** unauthorized
  - **404** PartitionID unknown
  - **413** too large
  - **507** no queue room
- Execution result comes back asynchronously as OCPRAS.1.0.0.PlatformActionEvent event with results in CPER

Drew Walton / Shubhada Pugaonkar

RAS API Open Source Demo

[KrishnaAnnam/redfish-dev-server: OCP RAS API Demo]



Microsoft demo

Arm demo

- Simulates: BMC that supports the RAS API, Data Center Analysis Framework
- Use the QR code to see the source code on GitHub

Shubhada Pugaonkar

RAS API - Vendor Analyzer Integration

Proof of Concept Implementation

1. Use [redfish-dev-server] at EndPoint(EP) and initiator end.
2. The analysis orchestrator integrates various vendor analyzers.
3. Build a vendor specific wrapper and integrate a black-box decoder and service action functionality.
4. Operator Policy Engine approves or overrides CPAD action.
5. Send CPAD back to EP for action.

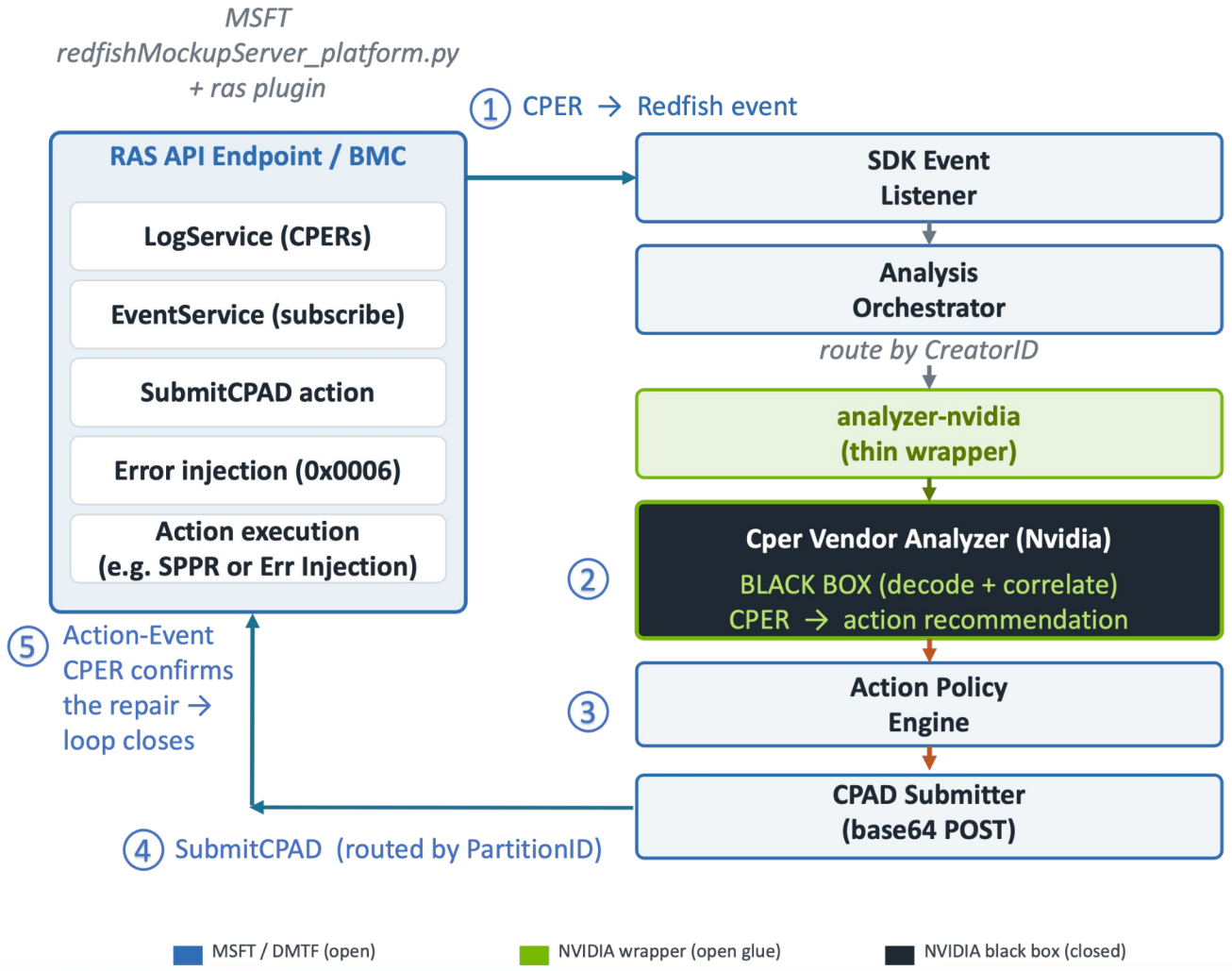
Flow

1. A CE DRAM error is logged as a CPER and delivered over Redfish.
2. Based on algorithm the vendor analyzer correlates a failing row.
3. It emits a repair CPAD (Soft Post-Package Repair or Err Injection for more errors).
4. The operator policy approves; the CPAD is routed back to the EP.
5. The BMC repairs the row and a follow-up Event CPER confirms it. The E2E round-trip flow complete.

Get Involved

•Try it on GitHub — microsoft/redfish-dev-server (OCP RAS API demo).

•See the live end-to-end demo in the booth.



Shubhada Pugaonkar

RAS API PLDM and Memory Analysis Demo

RAS API Demo

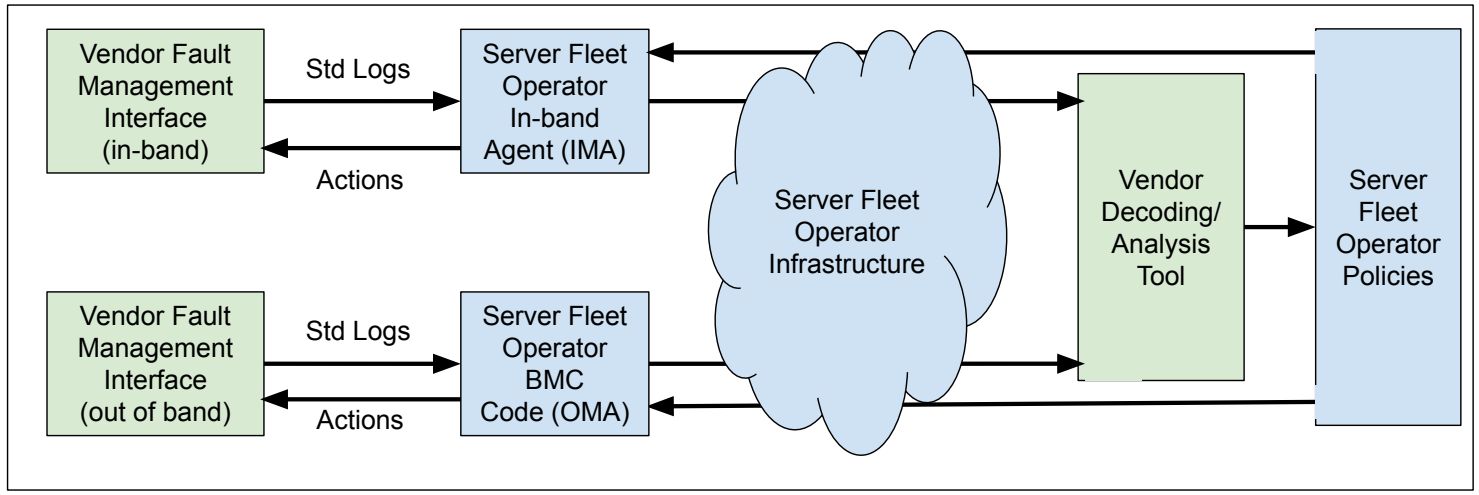
- Framework builds solution using concepts from HW Fault Management Requirements Document
- Demonstration of new mappings and interoperability

RAS API PLDM Demo: Arm

Simulates full data flow on Si that supports RAS API on PLDM

RAS API Memory Error Analysis Demo: Micron, Samsung & Microsoft

Simulates advanced memory error analysis from Micron & Samsung driving RAS actions



Come see the demo and talk to us in the Arm Booth

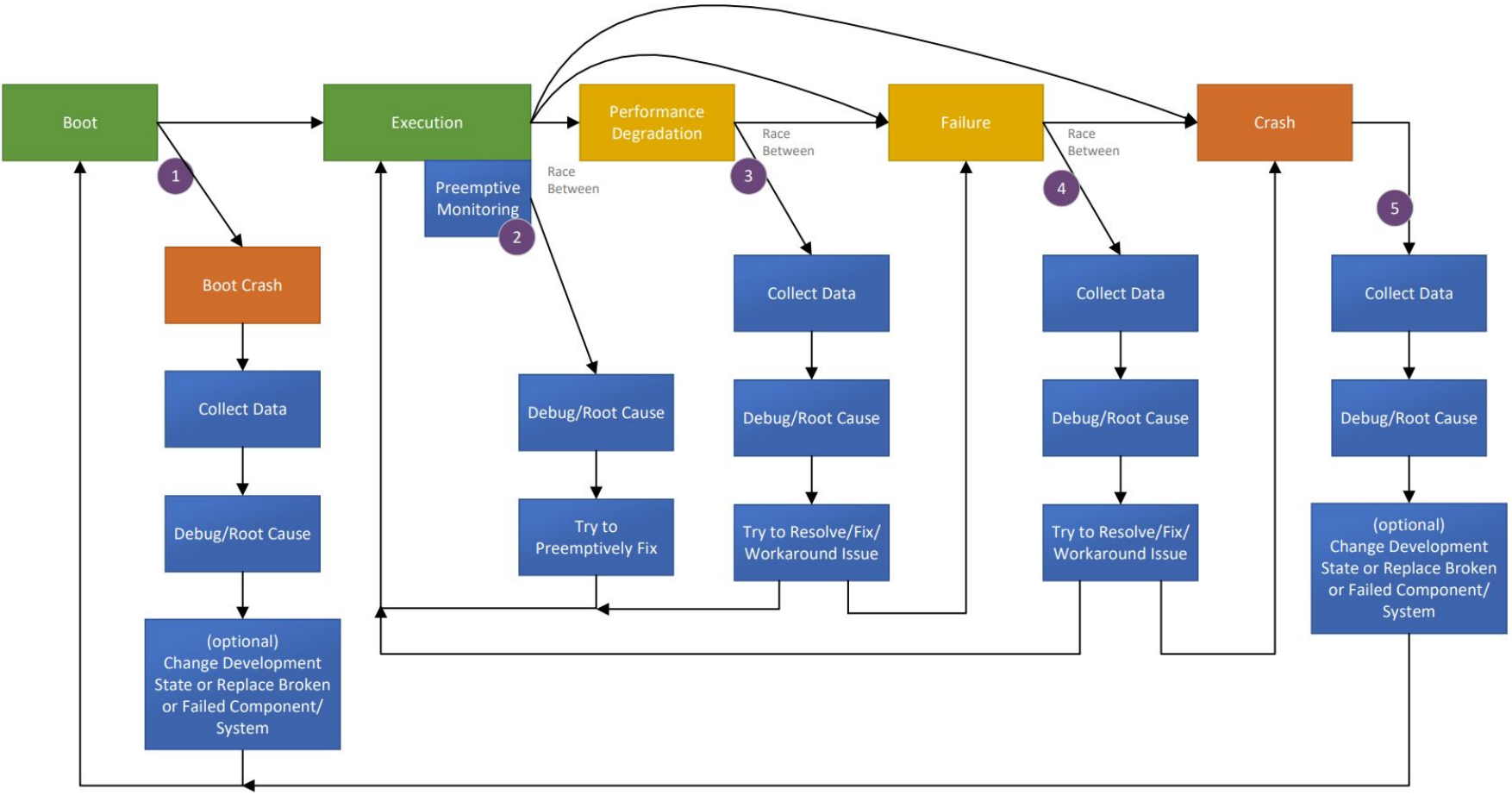
System Debug Workstream Progress

Marko Bartscherer

- Working on methods to accelerate debug of systems and components in data centers
- Upcoming spec focuses on providing non-sensitive debug information
  - Information that has no damaging effect to owner or customer
  - Transferred in a human auditable form
  - Allows information gathering in any scenarios
- Code Currency
  - State of the codebase (e.g. FW version) and configuration of the Node
  - JSON File exposed through Redfish
- Status Dump
  - State data and error codes from a system and component meant for triage
  - JSON File exposed through Redfish
- System Debug specification will be conformant with the OCP HWFM Framework

System Debug Workstream

Marko Bartscherer



Future work

- Generalization of specifications from closed CLAs (Crash Dump, Debug Dumps)
- Detection of performance degradation

System Debug WS meets weekly on Thursday 11am-12 PT

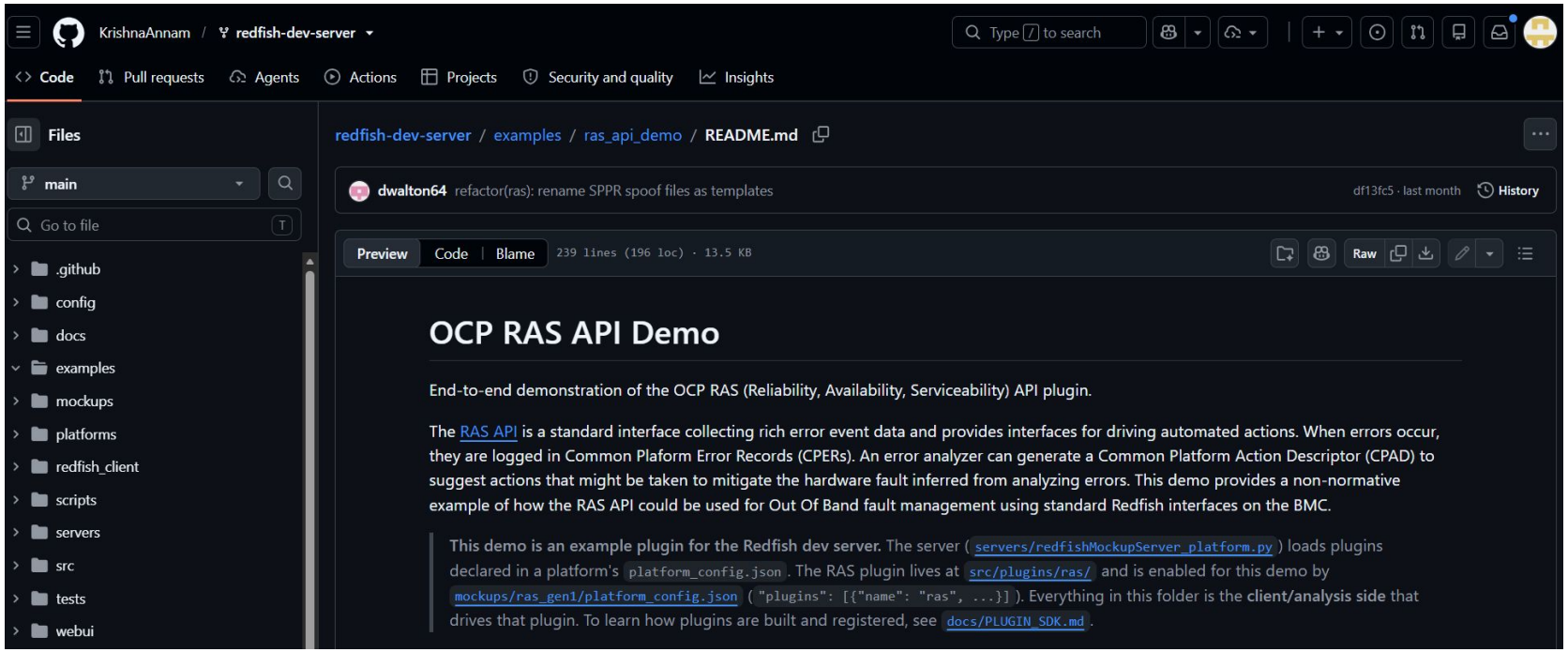
Call to Action - HWFM, FMFM, RAS API, Systems Debug

Drew Walton

- Get involved!
- Support RAS API (PLDM & Redfish) mapping standardization efforts at DMTF
- Join our meetings - Calendars are on the respective project Wiki pages below:
  - HWFM - https://www.opencompute.org/wiki/Hardware_Management/Hardware_Fault_Management
  - FMFM - https://www.opencompute.org/w/index.php?title=Hardware_Management/FMFM
  - RAS API - https://www.opencompute.org/w/index.php?title=RAS_API_Workstream
  - System Debug - https://github.com/opencomputeproject/OCP-HM-Hardware_Management/wiki/Wiki-%E2%80%90-System-Debug
- Subscribe to our mailing list:
  - HWFM - OCP-HWFaultMgt+subscribe@OCP-All.groups.io
  - FMFM - OCP-HWMgt-FMFM+subscribe@OCP-All.groups.io
  - RAS API - OCP-RAS-API+subscribe@OCP-All.groups.io
  - System Debug - OCP-system-debug+subscribe@OCP-All.groups.io

RAS API Open Source Demo

[KrishnaAnnam/redfish-dev-server: OCP RAS API Demo]



- Simulates: BMC that supports the RAS API, Data Center Analysis Framework
- Use the QR code to see the source code on GitHub

Agenda

Allotted time	Editor	Description
1/2 mins	Drew	(anchor) Intro to the session.
5 mins	Drew	About HWFM SP and WSs - stitch the big picture - draw connections.
5 mins	Drew	Talk about HWFM progress including Error Injection and Error Severities progress.
Transition 30sec	Drew	Introduce new co-lead Theo to briefly talk about FMFM work.
5 mins	Theodros	Walkthrough FMFM plans.
3 mins	Vilas	RAS API Intro Explains how RAS API is transport agnostic, extensible and based on standard CPERS/CPADs for heterogeneous fleet adoption. Routing CPERs and CPADs.
3 mins	Samer	How we collaborate with DMTF standards (PLDM, Redfish etc).
5 mins	Janusz	Discusses PLDM FileIO type-7 and extensibility to manageability
5 mins	Carlos	RAS API Redfish mapping overview
5 mins (+ 5 mins of buffer)	Shubhada + Drew	Discuss WIP RedFish PoC/Prototypes. Point people to github and demo in booth.
4 mins	Marko	System Debug WS update
1 min	Drew	End the session with a call to action.
Remainder	Everyone	Questions